

TABLE DES MATIÈRES

Modélisation de l'entité Agent dans un système FAIR : synthèse

Le problème

Le fil philosophique

Analyse des stratégies et de leurs alternatives

Conclusion : règles de décision et mesures de mitigation

Quand appliquer TPC

Mitiger les risques de TPC en PostgreSQL

Pourquoi PostgreSQL malgré ses limites

Le choix en une phrase

Modélisation de l'entité Agent dans un système FAIR : synthèse



Le problème



Un modèle de données FAIR pour la recherche hydrologique doit satisfaire une exigence forte : les données doivent être compréhensibles et réutilisables indépendamment du système qui les a produites. Un export CSV doit se lire sans documentation interne. Une métadonnée doit porter sa propre sémantique.

Ce principe se heurte à une difficulté de modélisation récurrente : plusieurs tables nécessitent de référencer un **agent producteur** dont la nature peut être fondamentalement différente selon le contexte. Un batch de transformation peut avoir été exécuté par un chercheur ou par un pipeline automatisé. Une responsabilité fonctionnelle peut porter sur une personne ou sur une organisation. Dans les deux cas, le modèle doit représenter cette dualité sans mentir sur la nature des entités impliquées et sans rendre les données illisibles à l'export.

Cette dualité a une forme logique qu'il vaut la peine de nommer. Un agent est soit un humain soit une machine, deux formes mutuellement exclusives dotées chacune de ses attributs : c'est ce que la théorie des types appelle une somme. Or le modèle relationnel repose sur l'opération inverse, le produit : une relation y est un sous-ensemble d'un produit cartésien de domaines, et une ligne est une conjonction de champs qui coexistent. Le relationnel exprime donc nativement la conjonction de faits, mais ne possède aucune construction native pour la disjonction de formes. Les trois stratégies qui suivent ne sont, de ce point de vue, que trois manières de simuler une somme dans un modèle qui ne connaît que le produit.

En modélisation relationnelle, trois stratégies canoniques répondent à ce type de problème. **TPH** (Table Per Hierarchy) fusionne tous les types dans une table unique avec un discriminant et des colonnes NULL selon le type. **TPT** (Table Per Type) introduit une table parente pour les attributs communs et des tables filles pour les attributs spécifiques. **TPC** (Table Per Concrete type) supprime toute table parente et laisse chaque type concret dans sa propre table, la relation polymorphique étant portée par un discriminant dans la table cliente.

Le choix entre ces stratégies ne se réduit pas à une question de performance ou de commodité d'implémentation. Il engage une prise de position sur ce que le schéma est censé dire sur le monde.

Le fil philosophique



Les grandes ruptures intellectuelles du XXe siècle en logique et en philosophie du langage décrivent structurellement pourquoi certaines solutions de modélisation sont condamnées à échouer.

Frege distingue le sens et la référence d'un terme. Un agent humain et un agent machine peuvent avoir la même référence fonctionnelle dans le système tout en ayant des sens radicalement différents. TPH ignore cette distinction en traitant deux sens distincts comme s'ils partageaient la même référence structurelle. Le discriminant implicite ne suffit pas : la signification de chaque ligne dépend d'une convention que la table ne rend pas visible.

Russell montre avec sa théorie des types qu'on ne peut pas mélanger impunément des objets de niveaux logiques différents dans le même ensemble sans créer des contradictions. TPH construit exactement cet ensemble russellien. TPT le corrige en apparence avec une hiérarchie explicite, mais postule une essence commune dont on verra qu'elle n'existe pas dans le domaine.

Hilbert avait espéré formaliser toute la vérité mathématique dans un système axiomatique complet et décidable. C'est l'ambition implicite de TPH et TPT lorsqu'ils tentent de mettre toute la sémantique dans le schéma PostgreSQL. **Gödel** a détruit ce programme : dans tout système formel suffisamment expressif, il existe des vérités qui ne sont pas démontrables à l'intérieur du système. Un schéma de données ne peut pas capturer toute la vérité sémantique du domaine. La bonne réponse n'est pas un schéma plus contraignant, c'est une partition consciente de la vérité entre le schéma, les données elles-mêmes, et la couche applicative.

Turing complète ce tableau en montrant que la décidabilité complète est impossible. Aucun schéma ne pourra jamais valider automatiquement toute la sémantique FAIR. Le pipeline de calcul doit par ailleurs être traité comme une entité de plein droit, d'une autre nature qu'une personne : le représenter comme une **Person** aux champs vides est une erreur conceptuelle avant d'être une erreur technique.

Liskov énonce en 1987 le principe de substitution : un objet d'un sous-type doit pouvoir remplacer un objet du parent sans altérer les propriétés qu'un programme tient du parent. On l'invoque souvent pour disqualifier TPT, mais il faut être précis sur ce qu'il contraint : il porte sur ce que le parent promet, pas sur ce que les enfants ajoutent. Qu'une machine n'ait pas d'ORCID et qu'une personne pas de numéro de série ne viole rien, ce sont des attributs propres aux enfants. Un parent mince, réduit à ce qui est réellement commun, une identité et le rôle d'exécutant, reste parfaitement substituable ; ce que le principe condamne, c'est un parent qui promet trop en logeant chez lui des attributs spécifiques à un seul type, ce qui est une faute de conception et non une propriété de TPT. TPC, lui, ne respecte pas ce principe, il le rend sans objet, puisqu'il supprime le supertype auquel on aurait substitué. Ce que TPC honore n'est donc pas le principe de substitution mais la préférence plus ancienne de Liskov, présente dès CLU, pour la composition plutôt que pour l'héritage : deux types concrets reliés par une référence étiquetée, sans hiérarchie parent-enfant.

Wittgenstein fournit les outils conceptuels les plus précis pour nommer le problème, et il faut distinguer ses deux périodes.

Le Wittgenstein du Tractatus pose que le langage est une image du monde et que toute vérité doit pouvoir être dite dans un langage formel. C'est la fondation philosophique de TPH. Mais le Tractatus reconnaît lui-même sa limite : la structure logique du langage ne peut pas être dite dans ce même langage, elle se montre. Un schéma ne peut pas garantir sa propre sémantique dans ses propres termes.

Le Wittgenstein des Recherches Philosophiques abandonne l'idée d'un langage idéal et introduit trois concepts décisifs. Les **jeux de langage** : le sens d'un terme est son usage dans une pratique concrète, pas une essence abstraite. "Agent humain" et "agent machine" ne sont pas deux variantes du même jeu de langage, mais deux jeux distincts avec des règles différentes, des participants différents, et des responsabilités épistémiques différentes. TPH les fusionne dans une table unique ; TPT postule un méta-jeu parent qui n'existe pas dans la pratique scientifique. Les **ressemblances de famille** : un concept n'est pas toujours défini par une propriété commune à tous ses membres, mais par un réseau de ressemblances qui se croisent sans se superposer entièrement. Un humain et une machine partagent un rôle fonctionnel sans partager d'essence. Vouloir les unifier dans une table avec une essence commune, c'est chercher une définition là où il n'y a qu'une ressemblance de famille. La **règle et son application** : suivre une règle n'est pas un acte mécanique, c'est une pratique sociale. Aucune contrainte SQL ne peut garantir que les métadonnées seront correctement remplies dans tous les cas futurs. La cohérence sémantique est une pratique communautaire, pas une propriété formelle du schéma. La règle doit donc être rendue visible dans les données elles-mêmes, pas enfouie dans des conventions implicites.

TPC est la traduction structurelle de ces leçons. Il reconnaît deux jeux de langage distincts sans prétendre les unifier. Il modélise une ressemblance de famille sans postuler d'essence commune. Il porte la règle dans les données via le discriminant `agentType`, lisible à l'export sans documentation externe.

Analyse des stratégies et de leurs alternatives



TPH est philosophiquement condamné dans un contexte FAIR. Son problème n'est pas les colonnes NULL (PostgreSQL les gère efficacement), c'est que la signification de chaque ligne dépend d'un discriminant dont la sémantique est implicite. Un export CSV mélange des colonnes qui ne coexistent jamais sans que le lecteur puisse le savoir sans documentation. La localité sémantique est perdue, une ligne ne s'interprétant plus sans documentation externe, ce que l'exigence de réutilisabilité de FAIR sanctionne. Les contraintes d'intégrité interne à chaque type (tel attribut obligatoire pour tel type) s'accumulent sous forme de règles implicites fragiles.

TPT est la pire des trois stratégies pour l'export. Elle répartit les attributs d'une entité sur plusieurs tables : un export de la table parente ne donne qu'un squelette incompréhensible sans jointure. Elle impose une jointure systématique à chaque lecture d'un agent complet, un coût cumulé réel. Elle crée un point de couplage entre tous les types via la table parente,

rigidifiant les migrations. Elle n'est en revanche légitime que si la table parente reste mince ; dès qu'on lui fait porter des attributs propres à un seul type, elle promet une communauté qui n'existe pas.

TPC a un coût technique concret et honnête. La FK native est perdue : PostgreSQL ne peut pas poser une contrainte de clé étrangère conditionnelle selon la valeur du discriminant. Les requêtes transversales sur tous les types nécessitent des `UNION ALL` verbeux. Le risque d'incohérence entre le discriminant et l'identifiant existe si l'API n'est pas le seul point d'écriture. Ces coûts sont réels mais circonscrits et mitigables. En contrepartie, TPC est la seule stratégie qui rend la sémantique auto-portée par les données : le discriminant `agentType` est lisible sur chaque ligne de chaque export, sans documentation externe. C'est ainsi qu'il résout en partie le problème de fond : il ne donne pas au moteur un vrai type somme, mais il place la disjonction, via `agentType`, là où un lecteur la trouve, sur la ligne elle-même.

Les **alternatives modernes** ne résolvent pas le problème de fond. JSONB déplace le schéma hors du schéma : la structure des attributs spécifiques devient un jeu de langage privé que seule l'application connaît, opaque pour tout outil externe. L'héritage PostgreSQL natif (`INHERITS`) a été conçu pour le partitionnement physique, pas pour la modélisation sémantique : les contraintes d'unicité, les FK et les index de la table parente ne couvrent pas les tables filles, rendant les garanties d'intégrité illusoires. Les tables de liaison polymorphiques intermédiaires n'apportent rien par rapport à TPC direct sinon une complexité supplémentaire. DuckDB et les triplestores RDF résolvent le problème différemment mais ne sont pas adaptés comme moteur de stockage transactionnel pour une API en production.

Conclusion : règles de décision et mesures de mitigation



Quand appliquer TPC



TPC est le bon choix lorsque les types cibles sont ontologiquement distincts (ils ne partagent pas d'essence commune, seulement un rôle fonctionnel dans un contexte précis), que leurs attributs spécifiques sont nombreux et structurellement différents, que la sémantique doit rester lisible localement, une ligne exportée devant s'interpréter seule, ce que FAIR motive sans en être la seule raison, que les types constituent un ensemble fermé ou lentement évolutif, et que le modèle utilise déjà le pattern polymorphique ailleurs.

TPC n'est pas adapté lorsque les types partagent une essence commune riche avec peu d'attributs spécifiques, lorsque les suppressions physiques sont fréquentes et qu'aucune politique de suppression logique n'est en place, ou lorsque des requêtes transversales massives et fréquentes sur tous les types sont critiques pour les performances.

Il existe un cas intermédiaire légitime : TPH localisé dans un contexte TPC global, lorsque deux types partagent un rôle fonctionnel identique avec des attributs communs dominants et peu d'attributs spécifiques, à condition que la contrainte applicative soit explicitement documentée et maintenue.

Mitiger les risques de TPC en PostgreSQL ↑

Six mesures techniques compensent la perte de la FK native et doivent être considérées comme non optionnelles.

Une contrainte `CHECK` sur le discriminant garantit que seules les valeurs connues et contrôlées sont acceptées. Un trigger `BEFORE INSERT OR UPDATE` sur chaque table cliente vérifie que l'identifiant référencé existe bien dans la table correspondant au type déclaré : c'est le substitut direct de la FK native. Un trigger d'interdiction des suppressions physiques sur les tables référencées force la suppression logique via un champ `archived_at` ou `status`, ce qui correspond par ailleurs à une exigence de traçabilité scientifique indépendante de TPC. Des vues encapsulent les `UNION ALL` nécessaires aux requêtes transversales une fois pour toutes, sans répétition dans chaque endpoint de l'API. Des commentaires PostgreSQL sur les colonnes `agentType` et `agentId` documentent le contrat polymorphique directement dans le schéma, lisible par tout outil d'inspection. Enfin, des requêtes de vérification d'intégrité exécutées périodiquement détectent toute référence orpheline qui aurait pu passer entre les mailles des triggers.

L'ensemble de ces mesures suppose une discipline d'écriture centralisée sur l'API : aucune écriture directe en base ne doit être possible pour les utilisateurs applicatifs. C'est la traduction concrète de la leçon de Gödel et de Turing : ce que le schéma ne peut pas garantir, la couche applicative doit le tenir explicitement, plutôt que de rêver un schéma qui porterait seul toute la vérité.

Pourquoi PostgreSQL malgré ses limites ↑

Les mesures de mitigation décrites ci-dessus sont nécessaires précisément parce que PostgreSQL ne résout pas le problème polymorphique nativement. Des moteurs alternatifs comme TypeDB le feraient : leur système de types somme permet de déclarer qu'un agent est soit une personne soit une machine comme une propriété du schéma lui-même, avec une garantie d'intégrité moteur que PostgreSQL ne peut pas offrir sur une FK en deux colonnes. Sur le plan de l'expressivité, c'est une réponse plus directe que la simulation relationnelle.

Sur le plan opérationnel, ces moteurs n'ont pas aujourd'hui la maturité nécessaire pour des données scientifiques à long terme : pas d'ORM stable, pas d'écosystème Django ou FastAPI, pas d'équivalent à TimescaleDB pour les séries temporelles, et une communauté trop restreinte pour garantir la pérennité sur 20 ou 30 ans.

Le choix de rester sur PostgreSQL n'est donc pas un renoncement philosophique. C'est une décision d'ingénierie lucide : accepter une limite structurelle connue du modèle relationnel, la compenser par des mesures explicites et auditables, et attendre que des moteurs philosophiquement plus corrects atteignent la maturité opérationnelle que des données de recherche exigent.

Le choix en une phrase



Un modèle de données FAIR qui adopte TPC massivement ne fait pas un compromis technique contraint. Il traduit honnêtement une réalité ontologique du domaine : des types distincts jouent parfois le même rôle fonctionnel sans partager d'essence, et cette vérité mérite d'être représentée lisiblement dans les données elles-mêmes plutôt qu'enfouie dans un schéma que seul le système d'origine sait lire.